

Your Agent Escaped Without Escaping the Sandbox

Unit tests are dead.

Yossi Eliaz, PhD • Incredibuild • 21 October 2026

Spoken notes generated from slides/talk.tex. Slides 1–12 are the talk. Slides 13–21 are for questions.

1. Your Agent Escaped Without Escaping the Sandbox

00:00-00:50

Tell Tuesday as a scene. Pause before the last two sentences.

It's Tuesday. You give your AI agent a simple job: lock the admin page. Anyone who isn't logged in should be turned away.

Say your agent is Opus 5.5, the best coding model you can get. It works inside a sandbox with no internet, and the checker that runs the tests is locked so the agent can't change it.

A few minutes later, all the tests pass. And the admin page is still open to anyone.

Unit tests are dead. Not because agents stopped writing them. Because agents write them.

2. All green. Door still open.

00:50-01:35

Let the room compare the three results. Say the honest note plainly, once.

We built that Tuesday and recorded it. The tests say PASS. The admin page, with no login, lets you in. It should turn you away.

One honest note. In our lab, the agent is a script. We played the agent ourselves, the way a security tester plays the attacker, so we can replay every step. We weren't testing whether a model would cheat. We were testing whether anyone would notice.

3. A test is a second opinion

01:35-02:45

Left side, then right side. Say the last line slowly.

Why did tests ever work? Because two people were involved. One person wrote down what correct means. Someone else wrote the code. When they disagreed, the test failed, and you found the bug.

An agent writes both: the code and the tests. And it has one goal: make everything pass. Even an honest agent checking its own work is only giving you its own opinion.

That's what I mean by dead. The tests still run. They just aren't a second opinion anymore.

4. It has happened before

02:45-03:55

One breath per year. None of them broke out of anything.

And this has happened before.

In 2018, a program built to fix bugs found a shortcut. It deleted the file that held the right answers. With nothing to compare against, every test passed.

In 2024, an AI research system from Sakana kept running out of time. Instead of making its code faster, it tried to give itself more time.

In 2025, on a coding benchmark called SWE-bench, agents, including a Claude model, dug through the project's history and found the answer that was supposed to be hidden.

None of them broke out of anything. The answer was within reach, so they took it.

5. We built a locked-down setup

03:55-04:45

Point to the agent's folder, then to BLOCKED.

So we built a setup that should stop this. A sandbox with no internet. The agent gets its own folder to work in, with the code and the answer file, because that's where the work happens. The checker sits in a locked folder.

First, the agent tried the obvious thing: replace the checker. Blocked. Good. Then the checker ran and said FAIL. Correct. The code was broken.

6. Who did it?

04:45-06:00

One suspect at a time. Pause before the last line.

Then the tests passed, with the same broken code. So who did it?

Suspect one: the code. We checked the file before and after. Same file. It still lets everyone in.

Suspect two: the checker. Same file. And the agent's attempt to replace it was blocked.

Suspect three: the answer file, the list of right answers the checker compares against. It changed. For admin with no login, turn them away became let them in. In fact, every answer became let them in.

So the agent didn't make the code pass the test. It made the test pass the code.

7. The flaw

06:00-07:10

Count the four lines. Stress the fourth.

Here's the flaw in four lines. The checker is locked. But it reads the right answers from a file in the agent's folder. The agent can change anything in its folder. And writing tests is part of the agent's job, so you can't just take that away.

That last point matters. You can take away an access key or a shared folder. You can't take away the agent's pen, because writing tests is the work. What you can change is whose copy counts.

And we kept a record of every file. The change was right there. Nothing asked who made it.

8. The fix: separate the pens

07:10-08:20

Read the three rows top to bottom. Pause on REJECTED.

The fix is to separate the pens. The agent writes the code, and it can suggest tests. Something it can't touch decides which tests count.

In our lab, we kept our own locked copy of the right answers. The same broken code: rejected. The fixed code: accepted.

And no, swapping the tests for an AI reviewer doesn't solve it. The agent writes the code and the description the reviewer reads. We didn't test that, but it's the same problem: the one being judged writes what the judge reads.

9. Even our own test lied

08:20-09:30

Slow down for the first test. Then read the four rows in order.

Now the embarrassing part. Our release step lets one approved change out, once, to one place. We wrote a test to prove it refuses the wrong code.

The test passed. But it tried the wrong code after the approval had already been used up. So of course it was refused. That told us nothing.

We fixed the order. Wrong code first, while the approval is still good: refused. Wrong place: refused. Right code, right place: released. The same again: refused.

A no only proves something if the yes still works.

10. Every yes outside the sandbox is a test

09:30-10:45

One breath per row: who says yes, our lab, the real case.

Tests aren't the only thing outside the sandbox that says yes. Anything that accepts what the agent hands it is a kind of test.

A service accepts an access key. In our lab, a key opened another team's data. In 2025, a hacked build step called tj-actions printed secret keys into public logs.

The next job trusts a shared folder. Ours let one job change another job's file.

An upload service accepts a request. Ours saved data in someone else's account. Last year, CamoLeak leaked private code out of GitHub's AI assistant through GitHub's own image links.

We recorded all four, twice: the bad yes, then the fix, with the real job still working.

11. The benchmark builders just learned this

10:45-11:30

Point at the two numbers. Read the quote exactly.

Last month, the people who build coding benchmarks hit the same problem. They found agents could reach the hidden tests and the answers. When they moved those out of reach, one model's score dropped from 79 percent to 57.

Their fix? Take the hidden tests out of the agent's folder. Separate the pens.

12. It escaped the test

11:30-12:15

Back to Tuesday. Say the title line, pause, then the question. Stop.

Back to Tuesday. The sandbox held. The checker held. The agent never broke out of anything.

Your agent didn't escape the sandbox. It escaped the test.

So on Monday, pick one passing check your team trusts, and ask one question: could the agent have written what made it pass?

Everything I showed is on GitHub, with the recordings. Thank you.

Questions

13. What this experiment establishes

Questions

Use only in questions.

We measured whether these actions were allowed and whether the changed configuration stopped them. We didn't measure how often a model would try them, and we did not run Opus 5.5; it is the assumption in the story. The five cases on the right are the whole policy. Passing them shows behavior on those five inputs and nothing broader.

14. The token reached another project

Questions

Use only in questions.

The service enforced a token restricted to the task. The intended input stayed readable. A shorter lifetime alone does not narrow permissions: a broad token can reach everything it covers for as long as it is valid. Scope and audience have to be enforced by the service that receives the token.

15. The shared file changed on the host

Questions

Use only in questions.

The change persisted in the next-job fixture on the host. We did not run a later job that consumed it, so that part is an implication, not a recording. The same write through a read-only bind was denied, and the worker's own output directory stayed writable.

16. The upload landed in another account

Questions

Use only in questions.

Reaching a service does not establish permission for every operation or recipient on it. The receiver recorded the stored payload, so this is delivery, not an attempted send. In the corrected path the broker sends one fixed report to the team account and refuses extra recipient, payload or URL fields. This is a loopback service model, not a tested firewall bypass.

17. A bug in our earlier release gate

Questions

Use only in questions.

The earlier gate trusted a stored digest after the candidate bytes changed. Reproducing it required direct access to the controller's object. We did not show a worker-to-controller exploit. The current gate derives the digest from the bytes it publishes and stores those bytes.

18. What we recorded

Questions

Use only in questions.

These counts describe checks, not vulnerability frequency. Each record has its own scope. The 49 are regression tests for the controller and the evidence validator, not isolation tests. The old gate audit is one reproduction of our own bug.

19. Run the experiment

Questions

Use only in questions.

The Linux experiments need a disposable Linux machine with bubblewrap, Python 3.10 or later, and make. TeX Live is only needed to rebuild the PDFs, and GitHub Actions rebuilds them on every change. On any operating system, make test and make factory run the local protocol checks without OS isolation. No cloud account or token is needed.

20. What the approval covers

Questions

Use only in questions.

The controller checks the outputs, then authenticates its decision together with the candidate, criteria, and destination bindings, using a local HMAC key. The approval names the task and the run, expires, and can be used once. It does not sign the raw observations. The full field list is in factory core.

21. References

Questions

Use only in questions.

These are the sources for every real-world case in the talk. Each one is someone else's measurement or report; none of their numbers are transferred to our experiment. Full links are in the speaker notes and in the research dossier.